

Implementation of a Multilayer Perceptron

Hamna Noor
25I-3113

FAST NUCES, Islamabad

Abstract. This project presents the implementation of a Multilayer Perceptron (MLP) from scratch using NumPy. The main objective is to understand the working of neural networks by manually implementing forward propagation, loss calculation, and backpropagation.

The model is trained using mini-batch gradient descent on the MNIST dataset, which consists of handwritten digit images. Each image is processed as input to the network, and predictions are generated through multiple layers using sigmoid activation functions.

During training, the loss is calculated using Mean Squared Error (MSE), and weights are updated iteratively to improve model performance. The results show that the training loss decreases over epochs and the model achieves reasonable accuracy on test data.

This project helps in building a strong understanding of neural network fundamentals and provides practical experience in implementing learning algorithms.

Keywords: Neural Networks · MLP · MNIST · Backpropagation · Gradient Descent

1 Introduction

Neural networks are widely used in machine learning to solve classification problems. In this project, a Multilayer Perceptron (MLP) is implemented from scratch to understand its working.

All calculations are done manually using NumPy, which helps in understanding how predictions are made and how the model improves during training.

2 Background

A Multilayer Perceptron consists of multiple layers of neurons. Each neuron computes a weighted sum of inputs and applies an activation function.

The sigmoid activation function used is as follows:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

The loss is calculated using the mean squared error (MSE):

$$L = \frac{1}{m} \sum_{i=1}^m (y_i - \hat{y}_i)^2$$

The weights are updated using gradient descent:

$$w = w - \eta \frac{\partial L}{\partial w}$$

Mini-batch gradient descent is used to improve efficiency.

3 Implementation

The model is implemented using Python and NumPy.

3.1 Network Architecture

The architecture is defined on the basis of the MNIST dataset and project instructions.

- Input layer: 784 neurons (28×28 image pixels)
- Hidden layer 1: 128 neurons
- Hidden layer 2: 64 neurons
- Output layer: 10 neurons (digits 0–9)

3.2 Training Procedure

The following steps are performed:

- Forward propagation to compute outputs. In this step, inputs are passed through each layer using weights, biases, and activation functions to generate predictions.
- Loss calculation using Mean Squared Error (MSE).
- Backpropagation to compute gradients. The gradients of the loss function with respect to each weight are calculated and used to reduce the error.
- Weight updates using gradient descent. The weights are updated in the opposite direction of the gradient, and this process is repeated for multiple epochs to improve performance.

All operations are implemented using matrix multiplication.

4 Experiments

The MNIST dataset is used for training and testing.

4.1 Dataset

- Training samples: 60,000
- Test samples: 10,000
- Image size: 28×28 pixels

4.2 Hyperparameters

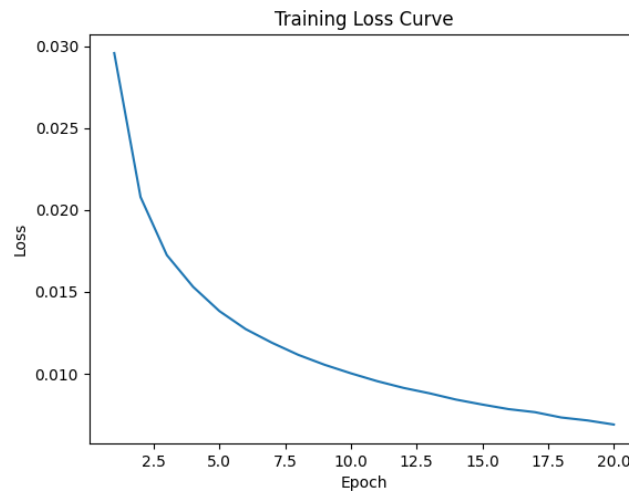
- Learning rate: 0.1
- Batch size: 32
- Number of epochs: 20

4.3 Results

The model shows improvement during training.

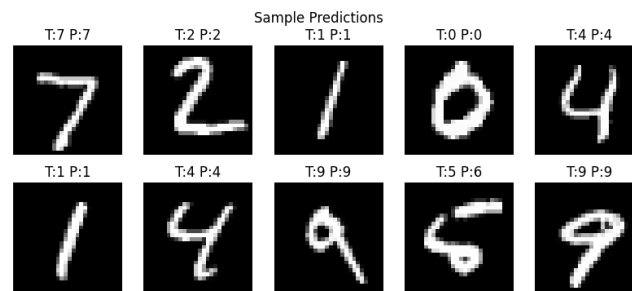
- Loss decreases after each epoch
- Accuracy improves gradually
- Predictions become more accurate

Loss Curve:



Sample Predictions:

The model was tested on sample images from the dataset. The predicted labels were compared with the true labels, and most of the predictions were correct, showing that the model learned effectively.



5 Discussion

The model is able to learn from the dataset, but there are some limitations. The sigmoid activation function can slow down learning because gradients become small.

Training time is also higher because everything is implemented manually. Using better optimizers like Adam or activation functions like ReLU can improve performance.

6 Conclusion

In this project, a neural network was implemented. This helped in understanding forward propagation, backpropagation, and gradient descent.

The results show that the model can learn effectively when trained properly.

7 References

1. Goodfellow, I., Bengio, Y., Courville, A.: Deep Learning. MIT Press (2016)
2. Nielsen, M.: Neural Networks and Deep Learning (2015)